

Mid-Level Software Engineer (Odoo) — Assignment

Time budget: 3–5 days (a few focused hours/day is fine — we care about quality of decisions, not hours logged) **Odoo version:** 17 or 18 (Community) — please state which version you used

1. Scenario

Build a custom module called `hr_training_request` that lets employees request external training/certifications, and routes that request through a manager and HR approval workflow.

We're specifically looking for how you handle **security, state transitions, and role-based UI** — not how many fields you can add.

2. Functional Requirements

2.1 Model: `hr.training.request`

- `employee_id` — Many2one to `hr.employee` (defaults to current user's employee record)
- `manager_id` — related/computed from `employee_id.parent_id` (read-only)
- `course_name` — Char, required
- `training_provider` — Char
- `start_date` / `end_date` — Date
- `cost` — Monetary/Float
- `justification` — Text
- `state` — Selection:
 - `draft` → `submitted` → `manager_approved` → `hr_approved`
 - `rejected` (reachable from `submitted` or `manager_approved`)
 - `cancelled` (reachable only from `draft` or `submitted`)

2.2 Security Groups

Create three groups (in addition to reusing existing `hr` groups where sensible):

- **Training Requester** (base employee access)
- **Training Manager Approver** (can approve/reject direct reports' requests)
- **Training HR Approver** (final approval, sees all company requests)

Define proper **record rules** so:

- Employees see only their own requests
- Managers see their own + their direct reports' requests
- HR Approvers see all requests (company-wide, respecting multi-company if you choose to support it)

2.3 State / Stage Transitions (role-gated)

- Only the request owner can **Submit** (draft → submitted) or **Cancel** (draft/submitted → cancelled)
- Only the employee's manager (or someone in the Manager Approver group) can **Approve/Reject** at the **submitted** stage
- Only someone in the HR Approver group can give final **Approve/Reject** at the **manager_approved** stage
- Trigger validation: **end_date** must be after **start_date**; block submission if **cost** is negative

2.4 Views

Build the standard view set and menu/action for **hr.training.request**:

- **Form view** — main data entry + workflow buttons (**statusbar** widget for **state** is expected)
- **List (tree) view** — key columns: employee, course name, dates, cost, state — with the state column visually distinguishable (decoration or badge widget)
- **Search view** — filters for each **state**, a "My Requests" filter (current user), a "My Team" filter (for managers), and **group_by** on **employee_id** and **state**
- A **menu item / action** wired up under a sensible parent menu (e.g., under Employees or its own top-level "Training" menu), with the action's domain/context adjusted so each role lands on a relevant default filter (e.g., HR Approver's menu opens to "Pending HR Approval")

Role-based behavior within these views:

- Action buttons (Submit, Approve, Reject, Cancel) must only be visible/enabled to the correct role at the correct stage — not just hidden in XML but properly guarded in Python too (never trust the client)
- Fields like **cost** and **justification** should become read-only once the request leaves **draft**
- HR Approvers should see an additional field/tab (e.g., internal HR notes) that Employees and Managers cannot see or edit — this should hold in **both** the form view and, if you surface it, the list view

2.5 Inherit **hr.employee**

- Add a computed field, e.g., **training_request_count**

- Add a smart button on the Employee form that opens that employee's training requests, filtered appropriately by the viewer's access rights
-

3. Technical Expectations

- Standard Odoo module structure (`__manifest__.py`, `security/`, `views/`, `models/`, `data/`)
 - `ir.model.access.csv` and record rules — don't rely on access rights alone for row-level security
 - Use `groups=` in views only as a UI convenience — actual enforcement must happen in `write()`/action methods too
 - No use of `sudo()` to silently bypass the access checks you just built — if you use it anywhere, comment why
 - Clean state machine — either a simple `state` Selection field with guarded button methods, or `mail.thread`/tracking if you want chatter logging of state changes (bonus, not required)
 - Include at least one demo data file so we can install and test without manually creating users/records
 - A short `README.md` explaining:
 - Your security group hierarchy and why
 - Any assumptions you made
 - What you'd improve with more time
-

4. Workflow Attachments

Three PNG wireframes to attach:

1. **Employee view** (draft stage) — fields editable, only Submit/Cancel buttons, "HR notes" tab locked
2. **Manager view** (submitted stage) — fields read-only, Approve/Reject buttons for the manager's own report, HR notes tab still locked
3. **HR Approver view** (manager-approved stage) — fields read-only, Final approve/Reject buttons, plus the extra HR notes field/tab only this role can see

Each includes the stage progress bar at top in the workflow that role acts, and a small callout banner explaining the role-based visibility rule being illustrated.

Training Requests / TR0001

Logged in as: Employee (Sarah Chen)

Draft

Submitted

Manager Approved

HR Approved

Submit

Cancel

Employee

Sarah Chen

Manager

James Patel

Course name

AWS Solutions Architect

Training provider

AWS Training

Start date

12/08/2026

End date

14/08/2026

Cost

£450.00

Description

HR notes

HR notes tab is not visible to employees or managers — only to HR Approvers.

Figure 1: employee form view

Training Requests / TR0001

Logged in as: Manager (James Patel)

Draft

Submitted

Manager Approved

HR Approved

Approve

Reject

Employee

Sarah Chen

Manager

James Patel

Course name

AWS Solutions Architect

Training provider

AWS Training

Start date

12/08/2026

End date

14/08/2026

Cost

£450.00

Description

HR notes

Manager sees Sarah's request because she is a direct report. Fields are read-only at this stage.

Figure 1: manager form view

The form displays a workflow diagram at the top with four stages: Draft (green), Submitted (green), Manager Approved (blue), and HR Approved (grey). Below the diagram are two buttons: 'Final approve' (green) and 'Reject' (red). The form fields are organized into two columns. The left column contains 'Employee' (Sarah Chen), 'Course name' (AWS Solutions Architect), 'Start date' (12/08/2026), and 'Cost' (£450.00). The right column contains 'Manager' (James Patel), 'Training provider' (AWS Training), and 'End date' (14/08/2026). Below these fields is a section for 'Internal HR notes (HR only)' with a text area containing 'Salary band eligible for training budget line B.'. At the bottom, there is a tabbed interface with 'Description' and 'HR notes' tabs. A note at the bottom states: 'HR Approver sees an additional "HR notes" field/tab, hidden from Employee and Manager roles.'

Draft → Submitted → Manager Approved → HR Approved

Final approve Reject

Employee
Sarah Chen

Manager
James Patel

Course name
AWS Solutions Architect

Training provider
AWS Training

Start date
12/08/2026

End date
14/08/2026

Cost
£450.00

Internal HR notes (HR only)
Salary band eligible for training budget line B.

Description HR notes

HR Approver sees an additional "HR notes" field/tab, hidden from Employee and Manager roles.

Figure 1: HR form view

5. Deliverables

- A GitHub repo (preferred) or a zipped module
- README as described above
- Optional but appreciated: a 2–3 min screen recording showing the workflow from three different user logins (Employee, Manager, HR)

6. Evaluation Criteria

Area	What we're looking for
Security correctness	Record rules and access rights actually restrict data — not just view-level hiding
State machine design	Clean, guarded transitions; no illegal state jumps possible via any route (UI, XML-RPC, shell)
Role-based UX	Right people see the right buttons/fields at the right time
Code quality	Idiomatic Odoo ORM usage, sensible model/field naming, no dead code
Inheritance handling	Clean <code>hr.employee</code> extension without breaking core views
Communication	README clarity, sensible commit history

We're not grading on visual polish or feature breadth — a smaller, correctly-secured module beats a feature-rich one with access holes.

7. Submission

Please send the repo link (or zip) along with your README to [contact email] by [deadline]. If anything in the spec is ambiguous, document the assumption you made in the README rather than guessing silently — we want to see your reasoning either way.